



Your AI should be a staff, not a search box.

An **open-source agentic operating system**: many purpose-built agents · on hardware you own · thinking with the AI you choose · reachable by the people you let in · through Telegram · on one subscription.

$$\text{OpenClaw} + \text{Docker} + \text{Telegram} = \text{Hatchabot}$$

the open-source agent runtime · one isolated container per agent · the front door everyone already has

THE FIRST QUESTION

“Can't ChatGPT or Claude already do this?”

They remember you now. They run tasks on a schedule. They read your Drive. For one person in one app they are excellent — and Hatchabot uses one of them as its brain. Four things change once the assistant has to serve a household.

Everyone gets agents

Sharing an assistant there means everyone signs in — their own ChatGPT account, or a paid Claude seat. Here the family messages a Telegram contact: no app, no login, no second subscription.

The memory is yours

Both remember you. Neither lets you open that memory, correct a wrong fact, back it up or take it elsewhere. Here it's a text file on your disk that outlives rebuilds and model changes.

A staff, not a generalist

One assistant answers your taxes, the homework and the condo board in one voice from one context. These are separate by design — own persona, model, data, people — and consult each other mid-task.

Your machine, scoped

Their tasks run in their cloud, with their tools, across your whole account. Each agent here gets exactly what its job needs: this folder, that repo, mail it may read but never send.

If you're the only one using it, the chat app is the better deal — polished, and no setup. Hatchabot is for several jobs, several people, and things you'd rather keep at home.

THE IDEA

The opposite shape

Many agents, each with a job

A kitchen helper, a tax advisor, a homework tutor, a scheduler that reads its own inbox. Made in a minute: a name, a paragraph, a bot token.

On a computer you own

Every agent is a container on your machine. Memory, credentials and history stay home. Pair it with a local model and nothing leaves at all.

With the AI you choose

Claude (subscription or API), Gemini or a local model — per agent, switchable live. You buy **inference** from a lab and nothing else: how the agent behaves is open-source code and files on your disk.

For the people you let in

Invite family by link or QR. Talk to one agent privately or together in a group room, with memory shared or personal — and everyone told which.

Through Telegram

The app everyone already has. Each agent is a real contact with a name and a face. Nothing to install, no accounts to create.

Run as a fleet

Not a chatbot but a control plane: health, usage, audit trail, backups, security posture, upgrades rolled out one agent at a time.

OpenClaw + Docker = Hatchabot. Every agent is a stock OpenClaw gateway — the open-source agent runtime — in its own Docker container with its own volume, bot and credentials. Hatchabot is OpenClaw *new and improved for a household*: many isolated installs, run as a fleet, by more than one person.

COMPARED

Side by side

A chat app is a place you go to talk to one assistant. Hatchabot is a staff of agents that live on your machine and come to you.

	CLAUDE.AI / CHATGPT	HATCHABOT
Shape	One assistant, one app, one context	A staff of agents with jobs, for a household
Who can use it	You; others need their own account, and a paid seat on Claude	Anyone you invite, on Telegram, with no account of their own
Memory	Kept by the provider — you can clear it, not read or move it	Text files on your disk: read them, fix them, back them up, take them with you
Acts on its own	Scheduled tasks, in their cloud, with their tools	Schedules, events, and requests from other agents — on your machine
Access to your things	The connectors they built, across your whole account	Per agent: one folder, one repo, one account — with limits you set
Choice of AI	That company's models	Claude, Gemini or a local model — per agent, switchable live
If you leave	Your history stays with them	Export it, move machines, change provider — the agents are yours

Same model underneath. Different shape: one assistant you visit, or a staff that lives at home and answers your whole family.

The chat apps are polished, need no setup, and their models are excellent — Hatchabot doesn't compete with the model, it uses one. Coding agents (Codex, Claude Code) are a different tool again: one agent, one repo, one task — and one of the things you can build here. Describes the consumer apps as of September 2026.

Inference. That's the whole list.

A chat app is a product built around a model, and whoever owns the model owns the product: how the agent behaves, what it remembers, which tools it may touch, what it costs next year — and whether it still exists next year.

They sell tokens

That is the entire dependency. A subscription or an API key, called by the unmodified vendor binary or API.

Swapping labs is a setting, not a migration: an agent moves between Claude, Gemini and a local model without being rewritten.

The community owns the rest

OpenClaw is the open-source runtime; Hatchabot is the open-source control plane. MIT, both.

How an agent thinks, what it remembers, what it may reach and who may talk to it are files on your disk and code anyone can read, fork and fix.

Which is the point

OpenAI shut down its Assistants API in Aug 2026 and retires Agent Builder in Nov. Products built on a vendor's harness went with them.

Agents built here outlive any lab's roadmap — and any lab's pricing decision.

Rent the intelligence. Own the machinery.

TWO DELIBERATE CHOICES

Claude Max and a front door people already have

1 plan

One subscription, the whole household

A Claude Max plan is a flat monthly fee. `claude setup-token` turns it into a credential Hatchabot keeps on your machine and hands to every agent — revocable, never a shared login.

The kids' tutor, the family planner, your own advisors: **everyone gets their own agents, and you pay for one plan.** Nobody else needs an AI account — they message the agents made for them.

20 bots

Telegram, Slack or Discord — no app to build

Free, on every platform, and bots are a first-class feature: BotFather makes one in a minute, and a free account carries about 20 identities. Slack and Discord work the same way, one small app per agent, and all three connect outward — nothing of yours is exposed. An agent can also have no messaging app at all and answer in the Hatchabot app.

One plan and a free front door: a household of agents that's affordable to leave on.

An operating system for agents

An agent is a container with a job, credentials and a memory of its own — and Hatchabot runs a fleet of them on a machine you control. Including agents you build for other people.

Made by describing it

An agent is a name, a paragraph of English and the accounts you attach. No code, no deployment, no glue — which is why the twentieth is worth making at all.

What you'd otherwise build

An always-on machine that can hold a client's mailbox for a year. One isolated tenant per client. A front door they already have — a Telegram contact, no AI account, no seat per person.

Then it compounds

The first agent takes an afternoon. The twentieth takes a minute — the infrastructure is already standing.

Agents are portable goods

Export one as a single file, hand it to someone else, and they run and adapt it. The persona, rules and accumulated training are the valuable part — and they're yours to keep, sell or give away.

Honestly: OpenAI and Anthropic are *very* extensible, and people ship real products on them. They sell the engine. Neither hands you the operating layer around many long-lived agents — tenancy, credentials on your own hardware, a human front door, one console for the fleet.

Two agents doing real work

A meeting scheduler anyone can email

It owns a Gmail account. Anyone — no Telegram, no login, no account of any kind — emails their availability and who is required or optional, and it negotiates by email with every invitee until a time works. It runs votes too: put a decision to a group and it drives the thread to an answer.

Built by describing it in English. Then a second agent was built to test it: a QA agent wired to the scheduler **both ways**, running regressions to check it still answers correctly.

The whole thing travels as one portable file.

Managing condo corporations

The manager's Gmail and the corporation's Drive attached to one agent — scoped to it, mail attachable with sending switched off. The job: board minutes, contractor quotes driven to a decision, files kept in order, issues tracked, the reserve fund watched, the books checked.

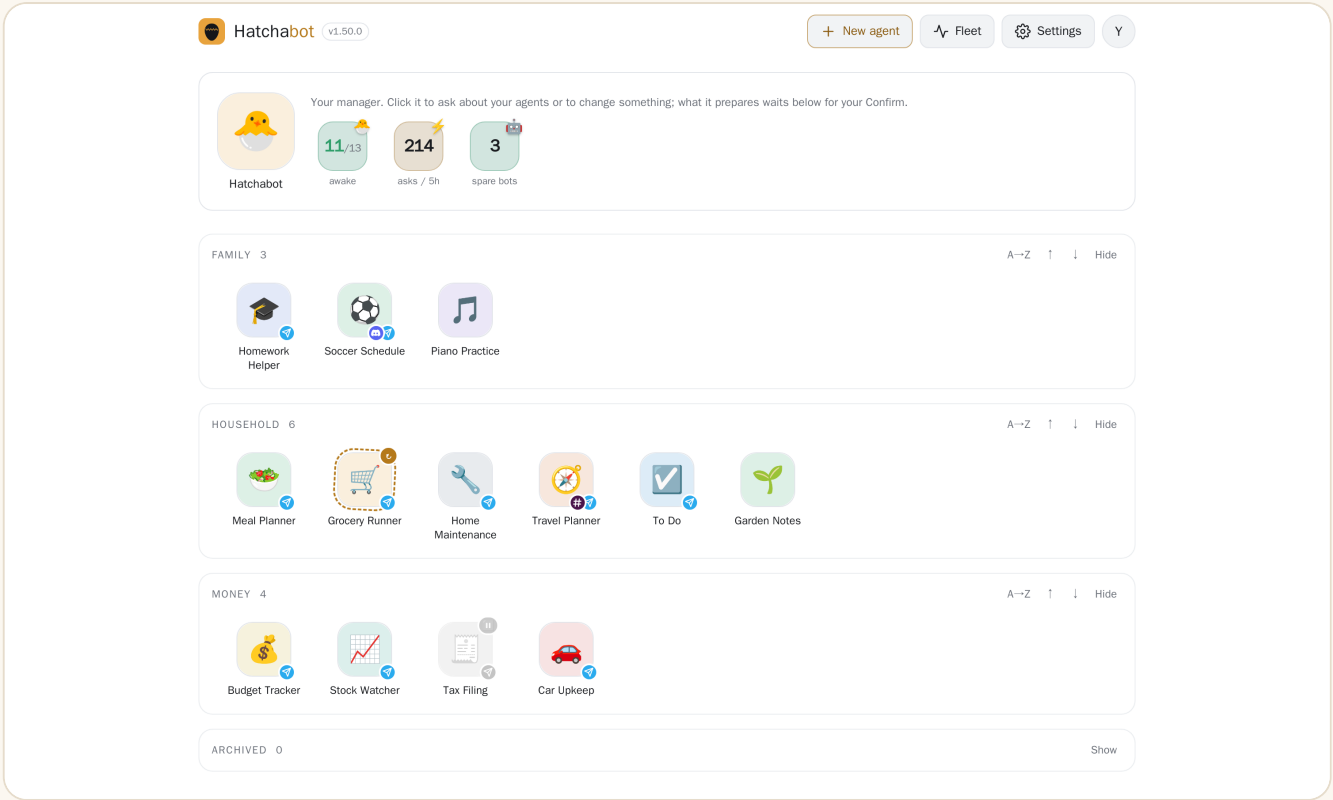
Then clone it for the next corporation: own container, own credentials, own memory — nothing mixes. Improve the master, push it to the copies.

One agent per client is the business.

Neither needed a line of code — and either can be handed to someone else as a file they run and adapt.

THE CONTROL PANEL

Your whole staff on one page



An example fleet — the names and activity are invented for this picture.

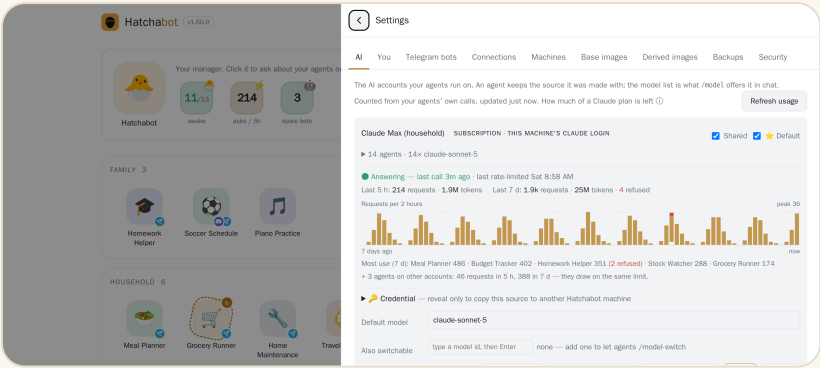
One page, the whole household

Every agent is an icon you name, colour and drag into groups. The icon carries its state: rebuilding, stopped, unread, and a mark per app that reaches it.

A manager you can talk to

The big icon is the Hatchabot agent, and the tiles beside it count what needs you. Ask it to make, fix, move or share an agent; it reads everything and changes nothing — each change comes back as a card you confirm.

Know what the plan is spending



FEATURES

What a staff can do that an assistant can't

CAPABILITY	IN PRACTICE
A new agent in a minute	A name and one paragraph → a running, remembering agent. Clone one that works, or import a template someone shared.
Many humans, one agent	Private one-to-one threads with the same agent, or a shared room. Invite by link or QR. Memory shared or personal.
A different brain per job	Claude, Gemini or a local model; switch live; classes retune a whole tier at once; migrate every agent off a source in one step.
Your accounts and files, scoped	Per agent: Gmail, Calendar, Drive and Sheets (with “read but never send”), read-only or writable folders, git repos it commits to — public ones need no key at all.
Work while you sleep	Scheduled tasks, and event-triggered ones where a zero-cost check decides whether to wake the model at all.
Agents that consult each other	Your investing agent asks your tax and legal agents mid-task, with scoped tokens, loop guards and rate limits.
Reachable where people are	Telegram, Slack or Discord — one, several or none. Each agent has its own bot; strangers get a code you approve.
A manager agent	Ask in plain words. It reads the fleet and proposes; every change is a card you confirm.

Each agent gets only what its job needs — which is what makes it safe to give them real work.

Operator-grade, because agents that matter need it

See what's going on

- Fleet health: probed, not assumed
- Usage per agent per day, tokens per hour
- An audit timeline of every change
- A daily security posture with diffs
- A legend showing every pending action

Change things safely

- Bulk actions by source, group, class or state
- Classes: model + source + runtime image, retuned in one place
- Runtime images: build a candidate, try it on **one** agent, promote or discard
- Snapshots before every edit; rollback on failed moves

Keep it yours

- Nightly backups: every volume, the registry, the key
- Restore one agent from any backup set
- Export and import as a single file
- Runners: place agents on a second machine
- All over a private Tailscale network — nothing opened to the internet

Everything an operator needs — and nothing a family member ever has to see.

What we believe an agent is

A thing you own

A container with a persona, operating rules and a memory — plain files you can read and edit:

SOUL.md · AGENTS.md ·
MEMORY.md

It survives restarts, rebuilds, model switches and moves between machines.

Memory is a file, not a chat log

The agent writes durable facts as it goes. Save a conversation to memory, download its whole history, or recover the context of a thread that got reset.

Nothing important is trapped in a transcript.

Isolated by construction

One container, one bot, one set of credentials per agent — given exactly the data its job needs and nothing else.

A daily posture check names every agent that has both a wide audience and a powerful capability.

Reversible, or honest about it

Snapshots before edits and rebuilds. Rollback on a failed move. A full audit timeline. Health probes that tell “listed as running” from “actually answering”.

An agent is a folder of files you own, not a session you visit — and one agent failing never takes the others with it.

COMPARED

OpenClaw, new and improved: what Hatchabot adds

Every Hatchabot agent **is** an OpenClaw gateway. Hatchabot adds the layer OpenClaw deliberately leaves out: many installs, run as a fleet, by more than one person.

One install per agent, so agents can't hurt each other

Plain OpenClaw is one process, one config, one workspace and credential set shared by every agent in it. Hatchabot gives each agent its own container, volume, bot token, tool policy and secrets.

Upgrades the same way

The runtime image is pinned and rolled out per agent, candidate first — never “upgrade the gateway and hope every agent survives”.

Lifecycle instead of hand-editing

Create, clone, rebuild, archive, restore, delete, move — with a snapshot before every change. In plain OpenClaw that's you, a shell and a directory.

More than one human

Owners, members, invites, pairing approvals, group rooms, and memory people are told is shared or private.

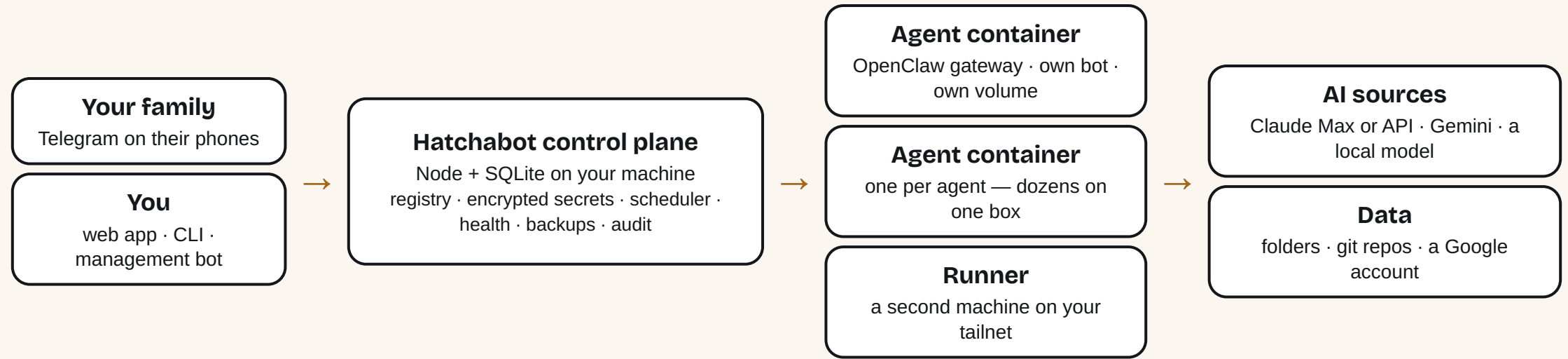
Credentials managed once

One setup-token serves every agent; Google accounts attach per agent; secrets are write-only.

If you run one agent for yourself, plain OpenClaw is fine. Hatchabot is for when there are several, they matter, and other people talk to them.

HOW IT WORKS

One control plane, one container per agent



Persisted

Each agent's `$HOME` is a Docker volume: persona, memory, sessions, tools. The runtime image is shared, versioned, and swapped under an agent without touching the volume.

Private by default

The web app lives on your machine; reach it from anywhere over Tailscale, never a port opened on your router. Telegram works from anywhere regardless.

Open source

TypeScript, MIT licensed, 978 automated tests and fifteen full audits. Tagged releases with notes; pre-built images for arm64 and amd64.

GET STARTED

Your first agent in 15 minutes

- 1 Make a Telegram bot.** In Telegram, open @BotFather, send `/newbot`, and keep the token.
- 2 Get Claude ready.** Install the Claude CLI, log in once, then run `claude setup-token` and keep that token too.
- 3 Install Hatchabot** on the computer that stays on:

```
$ bash -c "$(curl -fsSL https://hatchabot.com/install.sh)"
```
- 4 Open the app** at `http://localhost:8080`. The wizard takes both tokens and creates your first agent.
- 5 Say hi** on Telegram. It will still remember this conversation next year.

You need

- A Linux or macOS computer that stays on — a Mac mini, a home server, an old laptop
- A Telegram account
- A Claude subscription (Pro for a couple of agents, Max for a household) — or an API key, or a local model

Made to be easy to run

- The installer checks git, Docker and Node, and installs what's missing
- The agent image is pre-built — a pull, not a twenty-minute build
- `hatchabot doctor` checks everything and prints the fix



**For people who want more than one assistant
— shared with a household, on their own
machine, with the AI they choose.**

```
$ bash -c "$(curl -fsSL https://hatchabot.com/install.sh)"
```

hatchabot.com · github.com/hatchabot/hatchabot · MIT licensed